

17th May: Expo Router 101

1

-Jaani

Expo-router is built on top of react navigation.

Expo router is a file-based routing system, similar to Next.js.

index.ts → /

about.ts → /about

⇒ With Expo router, folders = routes

Any file inside app folder is treated as a route. But if you make a component, it will not be treated as a route.

profile/index.tsx → /profile

These are static routes

Dynamic Routes

A route whose path can change based on some value. eg id, username etc

Dynamic route folder structure

[2]

app/
└ product /
 └ [id].jsx

[] is used in dynamic route

eg [id].jsx

Nested Routes:

In Expo router, routes live inside a parent folder: e.g.

app /
└ index.jsx → /
└ profile /
 └ lol.jsx → profile / lol
 └ settings.jsx → profile / settings

Nested routes help us to organize things.

Dynamic Routes:

In dynamic routes, try keeping them inside a folder.

The name of folder and the value should be same.

⇒ return

<View>

<Text> Edit src/app </Text>

<Link href = { "profile / details" } About </Link>

<Link href = { " / user / 1234 " } Go to the
userId </Link>

<Link href = { " / username / jaani " } Go to username
page </Link>

<Link href = { " / docs / expo - routes " } Go to
Docs </Link>

</View>

)

}

Nested Dynamic Routes

⇒ Dynamic routes live inside nested folders

egg

app/

└ users/

└ [id]/

└ index.jsx

└ posts.jsx → / users / 42 /

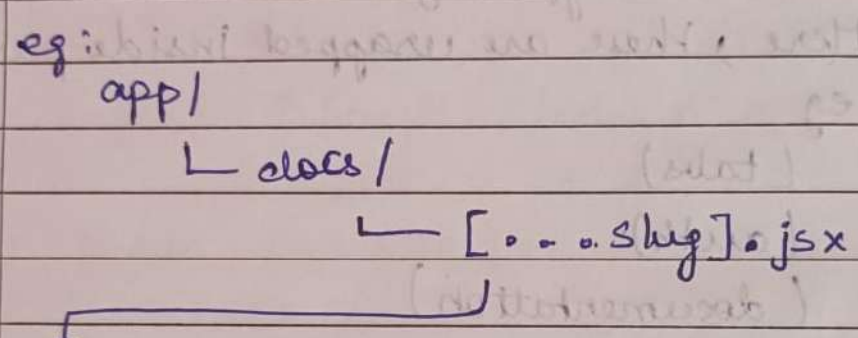
posts

Let's → To access these nested route, use

```
const { id } = useLocalSearchParams();
```

Catch-all dynamic route.

This captures multiple url segments using [...name].jsx



↓
This one file match with, /docs/react, /docs/react/hooks, /docs/react/hooks/useEffect etc

⇒ You can access this as ⇒

```
const { slug } = useLocalSearchParams();
```

⇒ Catch-all handles unlimited nested paths.

— layout.jsx

This wraps all the ~~new~~ children routes. It is the navigation wrapper for that folder.

Assignment : Read about { router, useRoutes }

⇒ layout uses stack navigation

Route groups :

They help organize routes and layouts without affecting the URL

Here, these are wrapped inside parenthesis

eg

(tabs)

(auth)

(documentation)

⇒ Main purpose of route groups are →

1. organize screens
2. apply layouts/navigation
3. separate app sections
4. keep URLs clean

Diff between layout and group router

→ layout controls navigation behavior like Stack/Tabs/Drawer

6

→ Route group only groups files together, but does not affect the URL.

Stack in Expo Router

⇒ stack works like a pile of screens stacked on top of each other.

To use it:

```
import { Stack } from "expo-router";
```

```
export default function AuthLayout() {
```

```
  return (
```

```
    <Stack>
```

```
      <Stack.Screen name = "index" />
```

```
    </Stack>
```

```
  )
```

#

Tabs



JS Tabs

Native Tabs

Custom tabs

```
eg: import { Tabs } from "expo-router"
```

```
export default function AuthLayout() {
```

```
  </Tabs>
```

```
}
```


7

Native tabs

Import {NativeTabs} from 'expo-router/unstable-native-tabs'

This is an experimental way to create real native bottom tabs instead of JS-based tabs.

Ios → native UITabBar

Android → native navigation components

⇒ These are less customizable
eg <NativeTab.trigger>

The access of all available tabs and current tabs is stored in state

All the screen options are stored in desc (descriptors)

To move between screens, use nav

Drawer:

18

In expo-router, a drawer is a side navigation menu from sides of the screen. It is setup in `_layout`

→ Write a blog on Expo Router vs React Navigation

→ Write a blog on how Instagram, whatsapp, uber and netflix would be built today using expo-router